



מדעי המחשב

בחינת מתכונת - תשפ"ב

משך הבחינה : 3 שעות ו-30 דק'.

מבנה השאלון ומפתח ההערכה: בשאלון זה שלושה פרקים

פרק ראשון – בפרק זה שלוש שאלות. יש לענות על השאלה הראשונה ולבחור אחת מבין שתי השאלות האחרות.

$(10X1) + (15X1)$ -- 25 נקודות

פרק שני – בפרק זה ארבע שאלות, ומהן עליך לענות על שתיים.

$(25X2)$ -- 50 נקודות

פרק שלישי – בשאלון הבגרות פרק זה מכיל שאלות בארבעה מסלולים שונים. ענה על שאלה אחת במסלול שלמדת. בשאלון הנוכחי שתי שאלות במסלול "תכנות מונחה עצמים", מהן עליך לענות על שאלה אחת.

$(25X1)$ -- 25 נקודות

סה"כ -- 100 נקודות

חומר עזר מותר בשימוש: כל חומר עזר חוץ ממחשב הניתן לתכנות.

נא לשמור על טוהר הבחינות.

שם: _____ כיתה: _____ מורה: _____

שלישי		שני				ראשון			פרק
9	8	7	6	5	4	3	2	1	שאלה
								✓	בחרתי ב-

ההנחיות בשאלון זה מנוסחות בלשון זכר ומכוונות לנבחנות ולנבחנים כאחד.

בהצלחה!

פרק ראשון (25 נקודות)

בפרק זה עליך לענות על שאלה 1 ובנוסף על אחת מבין השאלות 2 ו-3

שאלה 1 – חובה – 10 נקודות

נתונה הפעולה *Rev* המקבלת כפרמטר את *num* – שלם, ומחזירה שלם שסדר ספרותיו הפוך לסדר הספרות של *num*. למשל, עבור $num=7519$ יוחזר המספר 9157. שים לב: אין צורך לממש את הפעולה *Rev*.

כתוב את הפעולה *AnyRev* המקבלת כפרמטרים את *x* – שלם גדול מ-0 ואת *arr* – מערך שלמים גדולים מ-0. הפעולה תחזיר אמת אם ב-*arr* קיים מספר שסדר ספרותיו הפוך לסדר הספרות של *x*. אחרת, הפעולה תחזיר שקר. למשל, עבור $x=48$ והמערך $arr = \{9, 655, 84, 392\}$ הפעולה תחזיר אמת.

בפתרון שאלה זו יש לזמן את הפעולה *Rev*. הנח כי הספרה 0 לא מופיעה ב-*x* וגם לא במספרים השמורים במערך *arr*.

שאלה 2 – 15 נקודות – ענה על שאלה זו בדף נפרד

לאחרונה חלה עליה במקרי פציעה של תלמידים כגון שבירת יד בעת ביצוע פעילות ספורטיבית או נפילה מקורקינט נוסע וכו'. במחלקה **פציעה (Injury)** מוגדרות התכונות הבאות:

private int injType	סוג הפציעה (1- שבירת יד, 2- שבירת רגל, 3- חתך, וכך הלאה... עד 20- נקע באצבע)
private int causedBy	גורם הפציעה (1- פעילות ספורטיבית, 2- נפילה מקורקינט, וכך הלאה עד 7- קטטה)

בנוסף, במחלקה *Injury* מוגדרת פעולה בונה מלאה, ולכל אחת מהתכונות מוגדרות פעולה מאחזרת (*Get*) ופעולה קובעת (*Set*).

כתוב את הפעולה החיצונית *MCInjury* המקבלת כפרמטרים את *cb* – שלם שערכו בין 1 ל-7 ומייצג גורם פציעה ו-*arr* מערך פציעות. מבין הפציעות השמורות במערך *arr*, הפעולה תחזיר את המספר של סוג הפציעה הנפוץ ביותר ש-*cb* הוא הגורם שלו.

למשל, הפעולה תחזיר 3 (המספר של סוג פציעה חתך) עבור $cb=2$ אם מתוך כל הפציעות במערך חמש נגרמו בשל נפילה מקורקינט, כשבארבע מהן סוג הפציעה הוא חתך ובחמישית סוג הפציעה הוא שבירת יד.

הנח כי במערך *arr* אין הפניות ריקות (*null*) ו² עבור *cb* קיים סוג פציעה אחד נפוץ.

שאלה 3 – 15 נקודות – ענה על שאלה זו בדף נפרד

פיצרייה מציעה **מגש פיצה** אישי או משפחתי עם לכל היותר שתי תוספות מבין בצל(1), פטריות(2), תירס(3), זיתים(4).

מגש פיצה (Pizza) מאופיין על ידי גודל (*size*, שלם 1-פיצה אישית, 2-פיצה משפחתית), תוספת ראשונה (*add1* – שלם בין 0 ל-4, כאשר 0 משמעו אין תוספת ראשונה), תוספת שנייה (*add2*, שלם בין 0 ל-4, כאשר 0 משמעו אין תוספת שנייה). בנוסף, במחלקה *Pizza* מוגדרת פעולה בונה מלאה ולכל אחת מהתכונות מוגדרות פעולה מאחזרת (*Get*) ופעולה קובעת (*Set*).

א. (2 נק') במחלקה *Pizza* כתוב את הפעולה *SameAs* המקבלת כפרמטר את *p* – מגש פיצה אחר.

הפעולה תחזיר אמת אם גודלו של מגש הפיצה הנוכחי והתוספות שלו זהים לאלו של מגש הפיצה *p*. אחרת, הפעולה תחזיר שקר.

ב. (3 נק') במחלקה *Pizza* כתוב את הפעולה *GetPrice* המחשבת ומחזירה את מחיר מגש הפיצה, אם ידוע כי עבור פיצה אישית מחיר מגש הוא 30 ₪ ומחיר תוספת אחת הוא 3 ₪, ועבור פיצה משפחתית מחיר מגש הוא 60 ₪ ומחיר תוספת אחת הוא 6 ₪.

הזמנה (Order) מאופיינת על ידי שם מבצע/ת ההזמנה (*name*, מחרוזת) מגשי הפיצה שהוספו להזמנה (*ps*, מערך של 10 מגשי פיצה), מספר המגשים שהוזמנו בפועל (*count*, שלם).

ג. (3 נק') במחלקה *Order* כתוב פעולה בונה המקבלת פרמטר עבור שם מבצע/ת ההזמנה. בעת יצירת הזמנה, אין בה מגשי פיצה.

ד. (7 נק') במחלקה *Order* כתוב את הפעולה *AddPizza* המקבלת כפרמטר את *p* – מגש פיצה. הפעולה תוסיף את *p* להזמנה אם עדיין אין בה 10 מגשים וגם לא קיים בה מגש פיצה באותו גודל ועם אותן התוספות. הפעולה תחזיר אמת אם *p* נוסף להזמנה. אחרת, הפעולה תחזיר שקר.

פרק שני (50 נקודות)

בפרק זה עליך לענות על שתיים מבין השאלות 4, 5, 6 ו-7

שאלה 4 – 25 נקודות – ענה על שאלה זו בדף נפרד

א. (22 נק') כתוב את הפעולה **KToEnd** המקבלת כפרמטרים את ls – רשימה מקושרת של שלמים ואת k – שלם גדול מ-0. אם ברשימה ישנה חוליה שערכה גבוה מ- k הפעולה תעביר אותה אל סוף הרשימה. אם יש יותר מחוליה אחת שערכה גבוה מ- k , תועבר רק זו הקרובה ביותר לתחילת הרשימה. הפעולה תחזיר הפניה אל החוליה הראשונה ברשימה לאחר העדכון. **שים לב**: אין ליצור חוליה חדשה במקום זו שמעבירים, אלא להעביר את החוליה הקיימת.

להלן מספר דוגמאות עבור $k=3$:

דוג'	הרשימה לפני זימון הפעולה	הרשימה לאחר זימון הפעולה
1	$3 \rightarrow 2 \rightarrow 5 \rightarrow 97 \rightarrow 3 \rightarrow 1 \rightarrow 4 \rightarrow 8 \rightarrow \text{null}$	$3 \rightarrow 2 \rightarrow 97 \rightarrow 3 \rightarrow 1 \rightarrow 4 \rightarrow 8 \rightarrow \text{null}$
2	$6 \rightarrow 84 \rightarrow \text{null}$	$84 \rightarrow 6 \rightarrow \text{null}$
3	$1 \rightarrow 3 \rightarrow 2 \rightarrow \text{null}$	$1 \rightarrow 3 \rightarrow 2 \rightarrow \text{null}$

מומלץ לחשוב על התהליך ולשרטט אותו על דף לפני כתיבת הפתרון

ב. (3 נק') כתוב מהי סיבוכיות זמן הריצה של הפעולה **KToEnd**. נמק.

שאלה 5 – 25 נקודות – ענה על שאלה זו בדף נפרד

נתונה LST – רשימה מקושרת של שלמים בה מספר זוגי של חוליות, שערכיהן אינם שליליים. בנוסף נתון ARRS – מערך של מחסניות של תווים. חלק מהתווים השמורים במחסניות יוצרים יחד מחרוזת צופן.

להלן אופן בניית מחרוזת הצופן:

- * ערך החוליה הראשונה ב-LST מסמן אינדקס במערך ARRS בו נמצאת המחסנית בה שמור התו הראשון במחרוזת הצופן
 - * ערך החוליה השנייה ב-LST מסמן את מספר האיבר במחסנית שהינו התו הראשון במחרוזת הצופן
 - * ערך החוליה השלישית ב-LST מסמן אינדקס במערך ARRS בו נמצאת המחסנית בה שמור התו השני במחרוזת הצופן
 - * ערך החוליה הרביעית ב-LST מסמן את מספר האיבר במחסנית שהינו התו השני במחרוזת הצופן
- וכך הלאה..

למשל, מהרשימה הבאה

תו רביעי תו שלישי תו שני תו ראשון
 $\text{LST: } 5 \rightarrow 1 \rightarrow 0 \rightarrow 9 \rightarrow 8 \rightarrow 2 \rightarrow 0 \rightarrow 6 \rightarrow \text{null}$

ניתן ללמוד כי במחרוזת הצופן ארבעה תווים:

התו הראשון שמור במחסנית שנמצאת באינדקס חמש במערך ARRS והינו האיבר הראשון (Top) במחסנית
התו השני שמור במחסנית שנמצאת באינדקס אפס במערך ARRS והינו האיבר התשיעי במחסנית
התו השלישי שמור במחסנית שנמצאת באינדקס שמונה במערך ARRS והינו האיבר השני במחסנית
התו הרביעי שמור במחסנית שנמצאת באינדקס אפס במערך ARRS והינו האיבר השישי במחסנית

כתוב את הפעולה **Decipher** המקבלת כפרמטרים את LST – רשימה מקושרת של שלמים ואת ARRS – מערך מחסניות של תווים. לפי התיאור שלעיל, ועל סמך הנתונים השמורים ב-LST וב-ARRS הפעולה תבנה ותחזיר את מחרוזת הצופן.

הנח כי ¹ בכל אחד מתאי ARRS שמורה מחסנית לא ריקה של תווים, ² ב-LST מספר זוגי של חוליות ו-³ לכל זוג ערכים ברשימה LST קיימת מחסנית ב-ARRS ובמחסנית הזו קיים איבר (תו) מתאים.

```
public static int What(int num)
{
    if (num <= 9)
        return num + num;

    if (num <= 99)
        return num / 10 + num % 10;

    int d = num % 10;
    num = num / 100 * 10;
    return What(num + d);
}
```

א. (8 נק') נתונה הפעולה **What** המקבלת כפרמטר מספר שלם חיובי.

כתוב מעקב אחר הזימון הבא של הפעולה **What**

ציין בנוסף מה יהיה הפלט שיוצג:

```
int d = What(5804);
Console.WriteLine(d);
```

ב. (2 נק') כתוב את טענת היציאה של הפעולה **What**.

כלומר, הסבר בקצרה מה הפעולה מבצעת.

```
public static void Sod(Stack<int>st)
{
    if(!st.IsEmpty())
    {
        int x = What(st.Pop());
        if (!st.IsEmpty())
        {
            int y = What(st.Top());
            Sod(st);
            if(x > y)
                st.Push(x);
        }
    }
}
```

ג. (10 נק') נתונה הפעולה **Sod** המקבלת כפרמטר מחסנית של שלמים

חיוביים ומזמנת את הפעולה **What** שלעיל.

נתונה המחסנית ST הבאה:

ST(top)	84
	761
	4925
	580

בדף חדש ונפרד **כתוב מעקב** אחר הזימון בהמשך של הפעולה **Sod** עבור המחסנית ST בה ארבעה ערכים (העליון 84).

בכל שלב של המעקב יש לציין את ערכי המשתנים, ומהם האיברים במחסנית. ציין בנוסף מה יהיה הפלט שיוצג:

אין צורך לכלול במעקב את המתרחש בתוך הפעולה **What** אלא רק את הערך שהיא מחזירה.

Sod(ST);

if(!ST.IsEmpty())

Console.WriteLine("ST top is: {0}", ST.Top());

ד. (2 נק') כתוב את טענת היציאה של הפעולה **Sod**. כלומר, הסבר בקצרה מה הפעולה מבצעת.

ה. (3 נק') כתוב מהי סיבוכיות זמן הריצה של הפעולה **Sod**. נמק

שאלה 7 – 25 נקודות – ענה על שאלה זו בדף נפרד

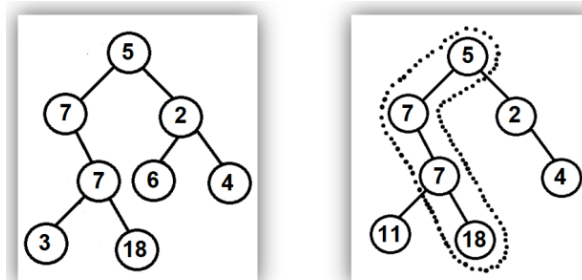
א. (22 נק') נגדיר "**מסלול משפחתי**" כמסלול המתחיל בשורש העץ ומסתיים בעלה כלשהו ויעבור מצומת אב אל צומת הבן שלו אם

לצומת בן יחיד וערכיהם שווים או כאשר לצומת שני בנים וערך הבן (בו עובר המסלול) שווה לסכום ערכי צומת האב והבן השני.

כתוב את הפעולה **RExist** המקבלת כפרמטר את t – עץ בינארי של שלמים גדולים מ-0.

הפעולה תחזיר אמת, אם ב-t קיים "**מסלול משפחתי**". אחרת, הפעולה תחזיר שקר.

למשל בעץ בצילום הימני המסלול התחום בנקודות עונה להגדרה של "**מסלול משפחתי**", אך בעץ בצילום השמאלי אין מסלול כזה.



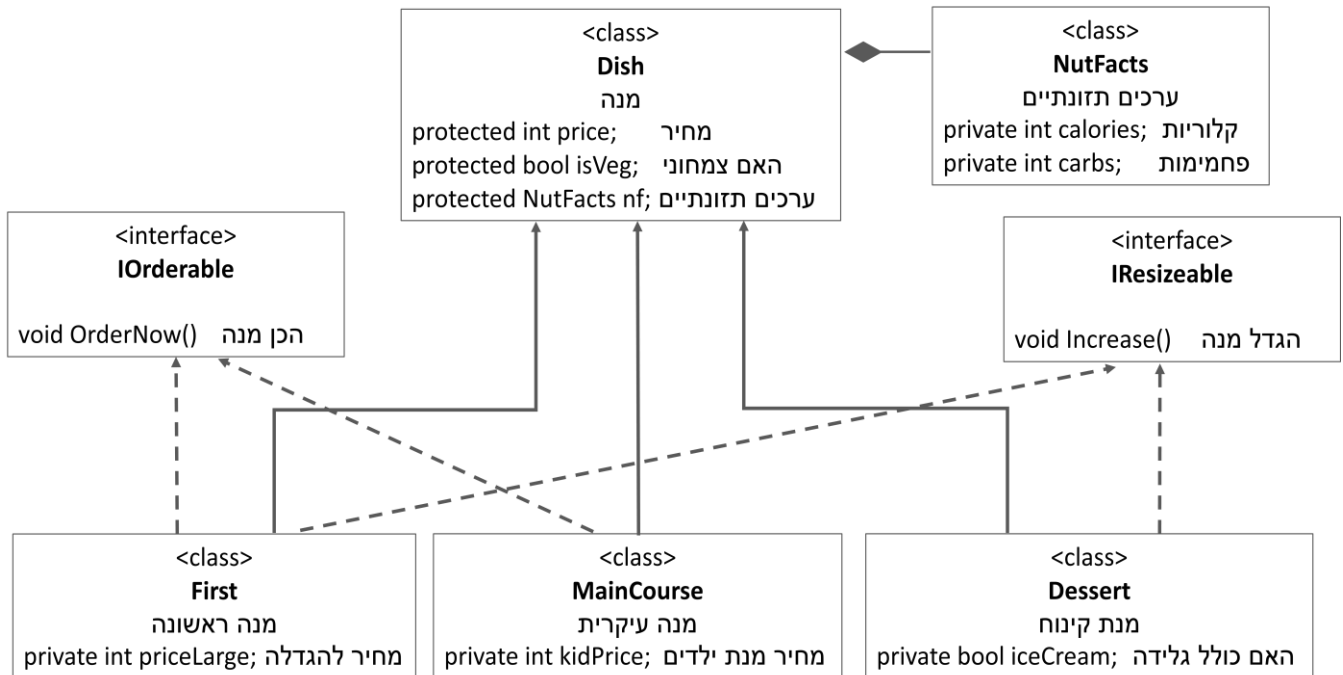
ב. (3 נק') כתוב מהי סיבוכיות זמן הריצה של הפעולה **RExist**. נמק.

פרק שלישי (25 נקודות)

בפרק זה עליך לענות על שאלה אחת מבין השאלות 8 ו-9

שאלה 8 – 25 נקודות – ענה על שאלה זו בדף נפרד

השרטוט הבא מתאר תרשים יחסים של מערכת לניהול מסעדה:



א. (9 נק') עבור כל אחת מהמחלקות **First**, **MainCourse**, **Dessert** כתוב את:

1. כותרת המחלקה
 2. כותרות כל הפעולות שחייבות להיות במחלקה, כולל הפעולות הבונות.
- הנח כי במחלקה **Dish** קיימת פעולה בונה מלאה (מקבלת פרמטר לכל תכונה).
אין צורך לממש את הפעולות ואין צורך להגדיר פעולות Get ו-Set

ב. (16 נק') לפניך קטע קוד.

עבור כל אחת מההוראות המסומנות בשורה ומספר, יש לקבוע אם היא תקינה או לא. אם ההוראה אינה תקינה יש להסביר מדוע.
 הנח כי ההוראות בשורות שאינן מסומנות תקינות.

```

Dish mc1 = new MainCourse(56, true, new NutFacts(120, 15), 42);
Dish mc2 = new MainCourse(42, false, new NutFacts(200, 35), 33);
Dessert ds = new Dessert(39, true, new NutFacts(280, 65), true);
IOrderable order1 = new IOrderable(); // -> line 1
IResizable size1 = ds;
IOrderable order2 = mc1; // -----> line 2
order2 = (IOrderable)mc2;
size1.SetPrice(31); // -----> line 3
order2.OrderNow(); // -----> line 4
order2.Increase(); // -----> line 5
((MainCourse)mc2).GetIsVeg(); // -----> line 6
mc1 = mc2; // -----> line 7
ds.GetCalories(); // -----> line 8
    
```

לפניך שלוש המחלקות A, B ו-C ובמודגש התכנית הראשית (main). קרא הקוד בעיון וענה על סעיפי השאלה בהמשך.

```
class A
{
    protected int f;
    private static int cA = 0;

    public A(int fa)
    {
        f = fa;
        cA++;
        Console.WriteLine("A {0} has {1}", cA, f);
    }

    public int GetF()
    {
        return this.f;
    }

    public virtual void Mult()
    {
        Console.WriteLine(this.f * cA);
    }

    public virtual void Print()
    {
        Console.WriteLine(this.f);
    }
}
```

```
class B : A
{
    private static int cB = 0;

    public B(int fb) : base(fb * 2)
    {
        B.cB++;
        Console.WriteLine("B {0} has {1}", B.cB, f);
    }

    public override void Mult()
    {
        Console.WriteLine(this.f * B.cB);
    }
}
```

```
class C : B
{
    private A a;

    public C(int fc, A a) : base(fc * 3)
    {
        this.a = a;
        int sum = a.GetF() + this.f;
        Console.WriteLine("C: {0}", sum);
    }

    public override void Print()
    {
        base.Print();
        a.Print();
    }
}
```

```
static void Main(string[] args)
{
    A[] arr = new A[3];
    arr[0] = new A(1);
    arr[1] = new C(2, new B(3));
    arr[2] = new C(4, arr[0]);

    for (int i = 0; i < arr.Length; i++)
    {
        if(arr[i] is B)
            Console.WriteLine("*****");
        else
            Console.WriteLine("#####");

        arr[i].Mult();
        arr[i].Print();
    }
}
```

א. (3 נק') שרטט עץ הורשה בו יופיעו שמות שלוש המחלקות וחיצים המציינים את הקשר ביניהן.

חץ רגיל יציין קשר של הורשה. ראש החץ יופנה אל מחלקת הבסיס.

חץ עם ראש מעוין יציין קשר של הכלה. ראש החץ יופנה אל המחלקה המכילה.

ב. (10 נק') שרטט את העצמים הנוצרים בתכנית הראשית (main) ולכל אחד מהם פרט את סוגו וערכי התכונות שלו בסיום הריצה.

ג. (12 נק') עקוב אחר התכנית הראשית (main) וכתוב מהו הפלט המוצג על המסך.